

Übersetzung und Analyse von Programmen

A stylized, low-poly illustration of a university building with large windows and a red abstract sculpture. In the foreground, there are two green trees and a large yellow flower with a blue center. The sky is blue with a large yellow sun and green geometric shapes.

Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl
Informatik, Universität Trier

TRIPLA mit Closures

- Lambda-Ausdrücke
- Alternative zu statischen Vorgängerverweisen

TRIPLA: Trierer Programmiersprache

Gegeben sei die Grammatik G:

```
E → let D in E
   | ID
   | ID ( A )
   | E ( A )
   | E AOP E
   | ( E )
   | CONST
   | ID = E
   | E ; E
   | if B then E else E
   | while B do { E }
   | lambda (V) { E }
```

A → E | A , E

D → ID (V) { E }
 | D D

V → ID | V , V

B → E | E RELOP E

ID: Bezeichner (fangen mit Buchstaben an und bestehen aus Buchstaben und Zahlen)

CONST: ganze Zahlen

AOP: arithm. Operatoren (+, -, *, /)

RELOP: Vergleichsoperatoren (==, !=, <, >)

Menge aller gültigen Programme: $LG(E) = \{w \mid E \rightarrow^* w\}$

Beispiele:

```
let main(f) {
  let inc(b) {
    f=lambda(x) {
      if (x==0) then 0
      else
        f(x-1) };
    f(b)
  } in inc(10); inc(5)
} in main(0)
```

```
let main(f) {
  let div(b) {
    if (b==0) then
      lambda(x) { 0 }
    else
      lambda(x) { x/b }
  } in f = div(10); f(20)
} in main(0)
```

Übersetzung von let in Lambda-Ausdruck

- $code (\text{let } id_1(id_{1,1}, \dots, id_{1,k_1})\{e_1\}$
...
 $id_m(id_{m,1}, \dots, id_{m,k_m})\{e_m\}$
 $\text{in } e) \rho = code (\text{lambda}(id_1, \dots, id_m) \{$
 $id_1 = \text{lambda}(id_{1,1}, \dots, id_{1,k_1})\{e_1\};$
 ...
 $id_m = \text{lambda}(id_{m,1}, \dots, id_{m,k_m})\{e_m\};$
 $e \}(0, \dots, 0)) \rho$

```
lambda (x, y) {  
  x+y+lambda (z) {  
    z+y+lambda (w) { w+y } (y) } (x) } (1, 2)
```

Klassifikation von Deklarationen von Parameter

local: Parameter, auf den es nur einen Zugriff im Rumpf der Funktion, aber nicht in einer enthaltenen Funktion gibt.

global: Parameter, auf den es einen Zugriff in einer enthaltenen Funktion gibt.

```
lambda (x, y) {
```

```
  x+y+lambda (z) {
```

```
    z+y+lambda (w) { w+y } (y)  } (x)  } (1, 2)
```

Klassifikation von Zugriffen auf Parameter

global: Zugriff auf **global** Parameter einer umgebenden Funktion

nonlocal: Zugriff auf **global** Parameter der aktuellen Funktionen

local: Zugriff auf **local** Parameter

Grundidee für die Implementierung:

- lokale Parameter werden auf dem Stack angelegt
- für globale Parameter wird eine Kopie im Heap angelegt
- für globale Parameter übergeordneter Funktionen werden Verweise im Heap angelegt, die auf die Kopien verweisen.

global

local

```
lambda (x, y) {
```

```
x+y+lambda (z) {
```

```
z+y+lambda (w) { w+y } (y) } (x) } (1, 2)
```

global: Zugriff auf **global** Parameter einer umgebenden Funktion

nonlocal: Zugriff auf **global** Parameter der aktuellen Funktionen

local: Zugriff auf **local** Parameter

Freie Variablen: local, nonlocal, global

- $\mathcal{M} = \{\text{local}, \text{nonlocal}, \text{global}\}$
- Ordnung auf $\mathcal{M}$: $\text{global} \sqsubseteq \text{nonlocal} \sqsubseteq \text{local}$
- $a \sqcup b = \min(a, b)$
- $A \sqcup B = \{ (x, a \sqcup b) \mid (x, a) \in A \text{ and } (x, b) \in B \}$

Freie Variablen

$$\text{free_vars}(\textcolor{blue}{k}) = \emptyset$$

$$\text{free_vars}(\textcolor{blue}{id}) = \{ (\textcolor{blue}{id}, \text{local}) \}$$

$$\text{free_vars}(\text{id}(\textcolor{gray}{e_1}, \dots, \textcolor{gray}{e_k})) = \text{free_vars } \textcolor{gray}{id} \sqcup \sqcup_{i=1}^k \text{free_vars } \textcolor{gray}{e_i}$$

$$\text{free_vars}(\textcolor{blue}{e}(\textcolor{blue}{e_1}, \dots, \textcolor{blue}{e_k})) = \text{free_vars } \textcolor{blue}{e} \sqcup \sqcup_{i=1}^k \text{free_vars } \textcolor{blue}{e_i}$$

$$\text{free_vars}(\text{lambda}(\textcolor{blue}{id_1}, \dots, \textcolor{blue}{id_k})\{\textcolor{blue}{e}\}) = \{ (x, \text{global}) \mid x \in \text{free_vars } \textcolor{blue}{e} \text{ and } x \notin \{\textcolor{blue}{id_1}, \dots, \textcolor{blue}{id_k}\} \}$$

$$\text{free_vars}(\textcolor{blue}{id}=\textcolor{blue}{e}) = \text{free_vars } \textcolor{blue}{id} \sqcup \text{free_vars } \textcolor{blue}{e}$$

$$\text{free_vars}(\textcolor{blue}{e_1} \text{ op } \textcolor{blue}{e_2}) = \text{free_vars } \textcolor{blue}{e_1} \sqcup \text{free_vars } \textcolor{blue}{e_2}$$

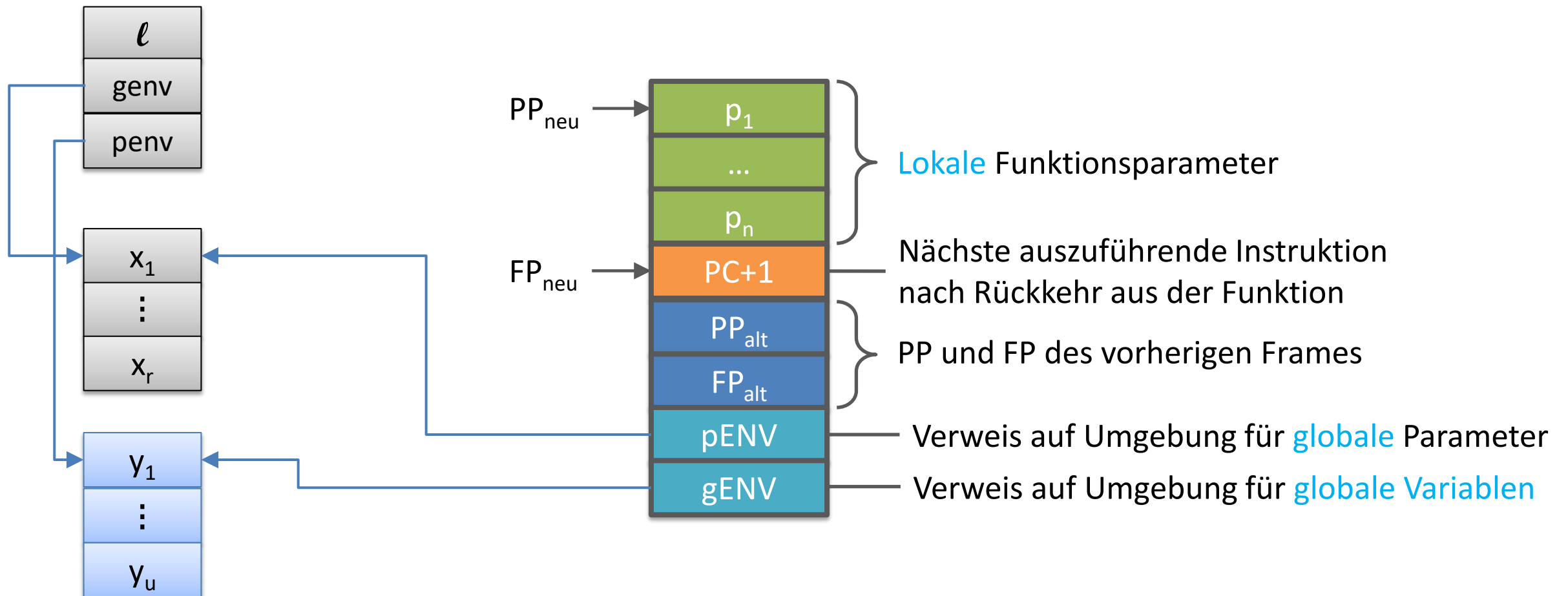
$$\text{free_vars}(\textcolor{blue}{e_1}; \dots; \textcolor{blue}{e_k}) = \sqcup_{i=1}^k \text{free_vars } \textcolor{blue}{e_i}$$

$$\text{free_vars}(\text{if } \textcolor{blue}{e_1} \text{ then } \textcolor{blue}{e_2} \text{ else } \textcolor{blue}{e_3}) = \text{free_vars } \textcolor{blue}{e_1} \sqcup \text{free_vars } \textcolor{blue}{e_2} \sqcup \text{free_vars } \textcolor{blue}{e_3}$$

$$\text{free_vars}(\text{while } \textcolor{blue}{e_1} \text{ do } \{ \textcolor{blue}{e_2} \}) = \text{free_vars } \textcolor{blue}{e_1} \sqcup \text{free_vars } \textcolor{blue}{e_2}$$

Abstrakte Maschine: Funktionen

Realisierung von Funktionen durch **Kellerrahmen** (stack frames):



Kodeerzeugung

(Kxx) $\text{code } (\text{lambda}(\text{id}_1, \dots, \text{id}_k)\{e\}) \rho =$

```

goto  $\ell_1$ ;
 $\ell_2$  : export  $i_1$  0
      ...
      export  $i_r$  r-1
      code e  $\rho_3$ ;
      return ;
 $\ell_1$  : loadref  $\rho(y_1)$ ;
      ...
      loadref  $\rho(y_u)$ ;
      closure  $\ell_2$  r u
    
```

$\text{loadref } (k, \text{nonlocal}) = \text{ploadref } k$

$\text{loadref } (k, \text{global}) = \text{gloadref } k$

where ℓ_1, ℓ_2 are new labels and $\text{fs} = \text{free_vars } e$
 and $[i_1, \dots, i_r] = \{i \mid (\text{id}_i, \text{global}) \in \text{fs}\}$
 and $[y_1, \dots, y_u] = \{x \mid (x, _) \in \text{fs} \text{ and } x \notin \{\text{id}_1, \dots, \text{id}_k\}\}$
 and $\rho_1 = \rho[(0, \text{local}) \setminus \text{id}_1] \dots [(k-1, \text{local}) \setminus \text{id}_k]$
 and $\rho_2 = \rho_1[(0, \text{nonlocal}) \setminus \text{id}_{i_1}] \dots [(r-1, \text{nonlocal}) \setminus \text{id}_{i_r}]$
 and $\rho_3 = \rho_2[(0, \text{global}) \setminus y_1] \dots [(u-1, \text{global}) \setminus y_u]$

indices of the parameters that
occur as global variables in e

Kodeerzeugung

(Kxx) *code* **id** $\rho = \text{load } k$ **where** $\rho(\text{id}) = (k, \text{local})$

(Kxx) *code* **id** $\rho = \text{pload } k$ **where** $\rho(\text{id}) = (k, \text{nonlocal})$

(Kxx) *code* **id** $\rho = \text{gload } k$ **where** $\rho(\text{id}) = (k, \text{global})$

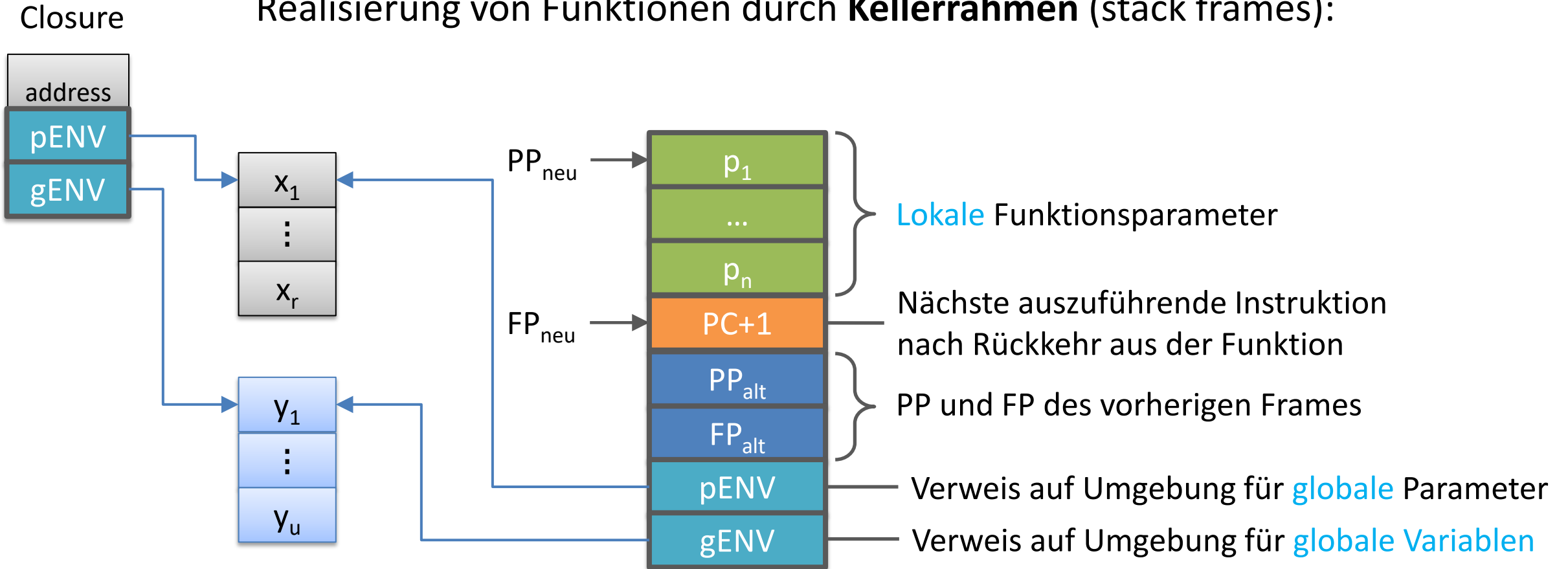
(Kxx) *code* (**id=e**) $\rho \text{ nl} =$ *code* **e** ρ ;
 store k ;
 load k **where** $(k, \text{local}) = \rho(\text{id})$

(Kxx) *code* (**id=e**) $\rho \text{ nl} =$ *code* **e** ρ ;
 pstore k ;
 pload k **where** $(k, \text{nonlocal}) = \rho(\text{id})$

(Kxx) *code* (**id=e**) $\rho \text{ nl} =$ *code* **e** ρ ;
 gstore k ;
 gload k **where** $(k, \text{global}) = \rho(\text{id})$

Abstrakte Maschine: Funktionen

Realisierung von Funktionen durch **Kellerrahmen** (stack frames):



Abstrakte Maschine: Ausdrücke

```
store k :      STACK[PP+k] = STACK[TOP]
                TOP = TOP - 1
                PC = PC + 1

load k :      STACK[TOP+1] = STACK[PP+k]
                TOP = TOP + 1
                PC = PC + 1

pstore k :    STACK[FP+3][k] = STACK[TOP]      // penv
                TOP = TOP - 1
                PC = PC + 1

pload k :     STACK[TOP+1] = STACK[FP+3][k]    // penv
                TOP = TOP + 1
                PC = PC + 1

gstore k :    STACK[FP+4][k] = STACK[TOP]      // genv
                TOP = TOP - 1
                PC = PC + 1

gload k :     STACK[TOP+1] = STACK[FP+4][k]    // genv
                TOP = TOP + 1
                PC = PC + 1
```

Kodeerzeugung

(Kxx) $code(\lambda(id_1, \dots, id_k)\{e\}) \rho =$

```

goto  $\ell_1$ ;
 $\ell_2$  : export  $i_1$  0
      ...
      export  $i_r$  r-1
      code e  $\rho_3$ ;
      return ;
 $\ell_1$  : loadref  $\rho(y_1)$ ;
      ...
      loadref  $\rho(y_u)$ ;
      closure  $\ell_2$  r u
  
```

$loadref(k, nonlocal) = ploadref\ k$
 $loadref(k, global) = gloadref\ k$

where ℓ_1, ℓ_2 are new labels and $fs = free_vars\ e$
 and $[i_1, \dots, i_r] = \{i \mid (id_i, global) \in fs\}$
 and $[y_1, \dots, y_u] = \{x \mid (x, _) \in fs \text{ and } x \notin \{id_1, \dots, id_k\}\}$
 and $\rho_1 = \rho[(0, local) \setminus id_1] \dots [(k-1, local) \setminus id_k]$
 and $\rho_2 = \rho_1[(0, nonlocal) \setminus id_{i_1}] \dots [(r-1, nonlocal) \setminus id_{i_r}]$
 and $\rho_3 = \rho_2[(0, global) \setminus y_1] \dots [(u-1, global) \setminus y_u]$

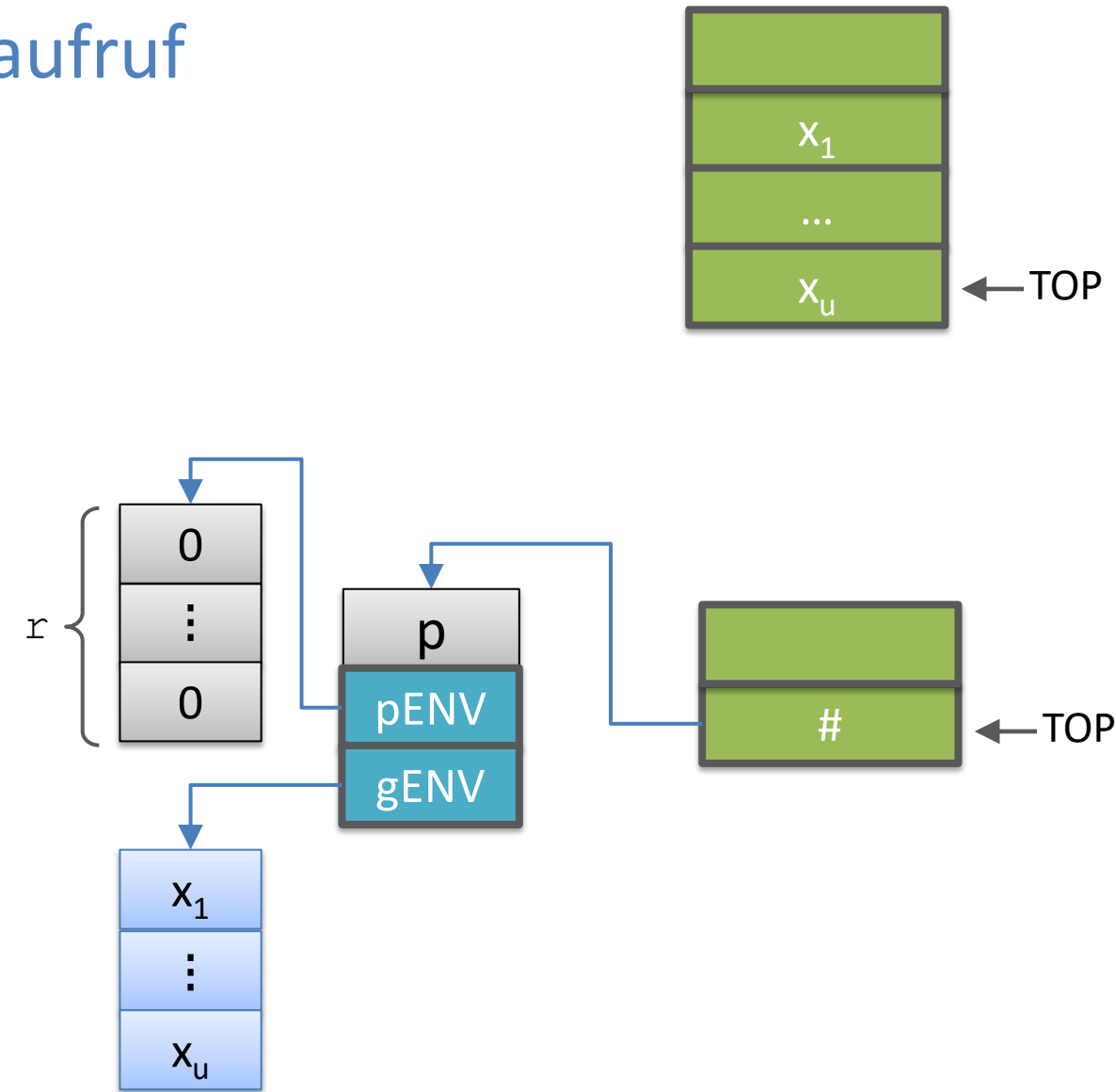
indices of the parameters that
 occur as global variables in e

Abstrakte Maschine

Funktionsaufruf

```

closure p r u: penv=[]
    for i=0 to u-1: penvi=0
    genv=[]
    for i=0 to u-1:
        genv[i]=STACK[TOP-u+i]
    clo = new Closure(p,penv,genv)
    TOP = TOP - u
    STACK[TOP] = clo
    PC = PC + 1
  
```



Kodeerzeugung

$(Kxx) \text{ code } (e(e_1, \dots, e_k)) \rho =$

- $\text{code } e \rho$
- $\text{code } e_1 \rho ;$
- $\dots$
- $\text{code } e_k \rho ;$
- $\text{invoke } k$

Abstrakte Maschine

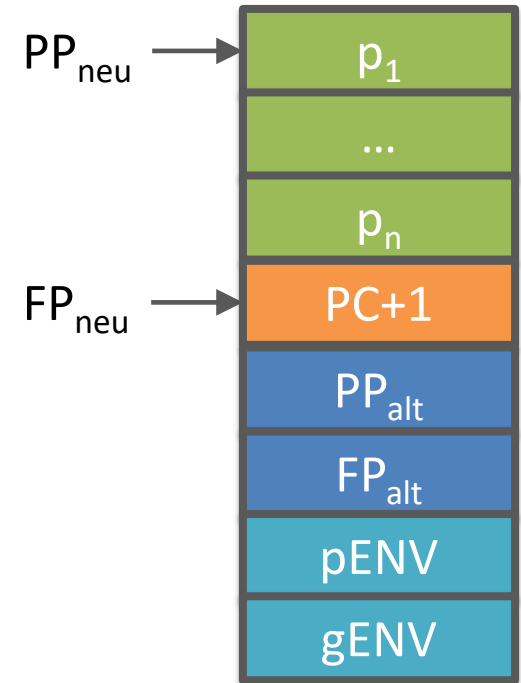
Funktionsaufruf

```

invoke n:      clo = HEAP[STACK[TOP-n]]
                  STACK[TOP] = PC+1
                  STACK[TOP+1] = PP
                  STACK[TOP+2] = FP
                  STACK[TOP+3] = clo.penv
                  STACK[TOP+4] = clo.genv
                  PP = TOP - n + 1
                  FP = TOP + 1
                  TOP = TOP + 5
                  PC = clo.address
  
```

```

return :      res = STACK[TOP]
                  TOP = PP-1
                  PC = STACK[FP]
                  PP = STACK[FP+1]
                  FP = STACK[FP+2]
                  STACK[TOP] = res
  
```



```
lambda(a, b){
```

$G = \{ a, b \}$

```
  lambda(main, ident) {
```

$G = \{ a, b \}, L = \{ \text{main}, \text{ident} \}$

```
    main = lambda (f, c) {
```

$G = \{ f, a, c \}, N = \{ b \}$

```
      lambda(g) {
```

$N = \{ f, g, a, c \}, L = \{ b \}$

```
        g = lambda(x, y) {
```

$L = \{ a, c, x, y, f, g \}$

```
          if x>a then f(g)(x-1,y) else c
```

```
        };
```

```
        g(a, b)
```

```
      }(-99)
```

```
    };
```

```
    ident = lambda(h) {
```

$L = \{ h \}$

```
      h };
```

```
    main(ident, a+b)
```

```
  }(-99,-99)
```

```
}(9, 0)
```

Klassifikation

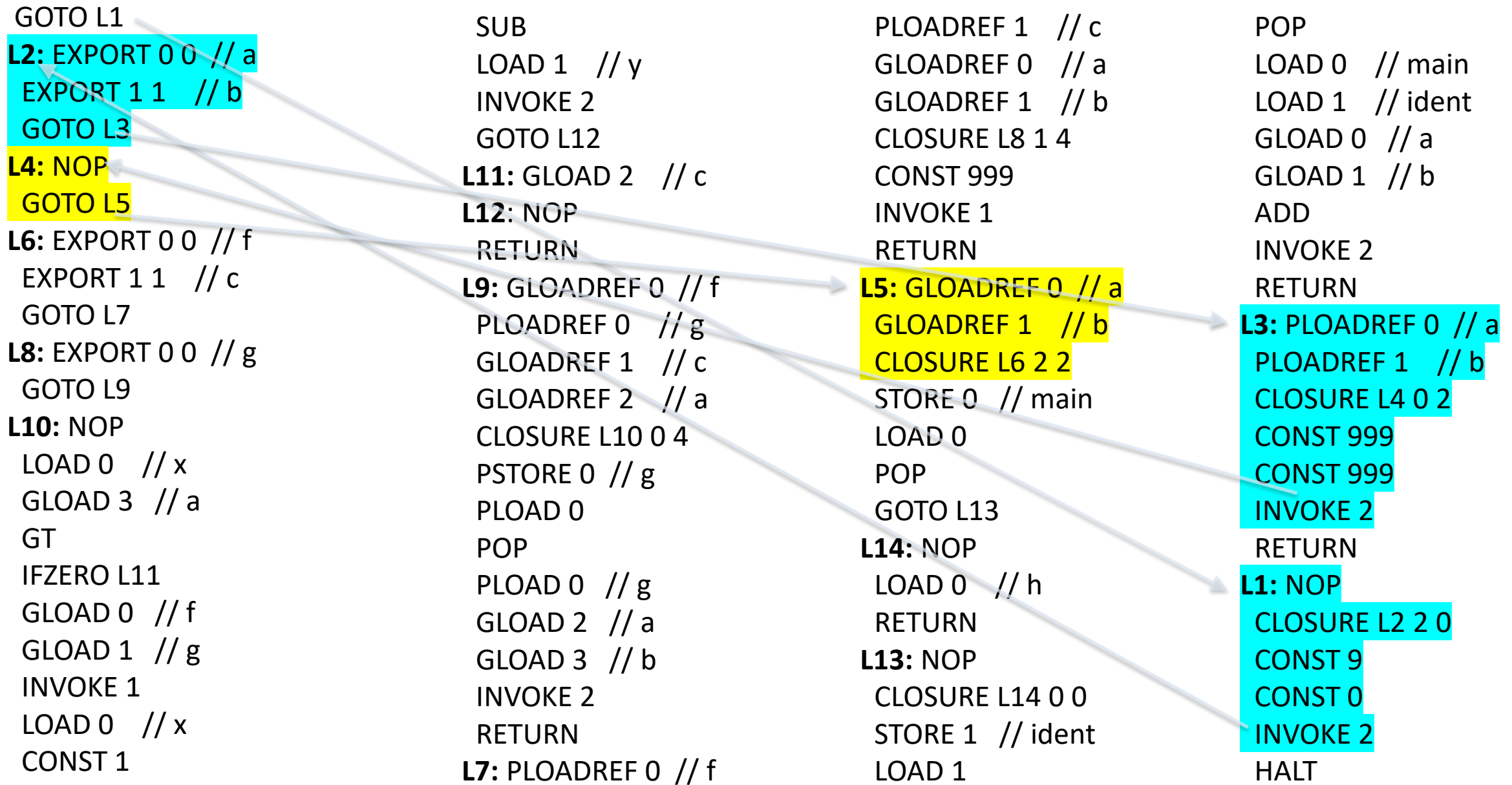
- local
- nonlocal
- global

GOTO L1
L2: EXPORT 0 0 // a
EXPORT 1 1 // b
GOTO L3
L4: NOP
GOTO L5
L6: EXPORT 0 0 // f
EXPORT 1 1 // c
GOTO L7
L8: EXPORT 0 0 // g
GOTO L9
L10: NOP
LOAD 0 // x
GLOAD 3 // a
GT
IFZERO L11
GLOAD 0 // f
GLOAD 1 // g
INVOKE 1
LOAD 0 // x
CONST 1
SUB
LOAD 1 // y

INVOKE 2
GOTO L12
L11: GLOAD 2 // c
L12: NOP
RETURN
L9: GLOADREF 0 // f
PLOADREF 0 // g
GLOADREF 1 // c
GLOADREF 2 // a
CLOSURE L10 0 4
PSTORE 0 // g
PLOAD 0
POP
PLOAD 0 // g
GLOAD 2 // a
GLOAD 3 // b
INVOKE 2
RETURN
L7: PLOADREF 0 // f
PLOADREF 1 // c
GLOADREF 0 // a
GLOADREF 1 // b
CLOSURE L8 1 4

CONST 0
CONST 99
SUB
INVOKE 1
RETURN
L5: GLOADREF 0 // a
GLOADREF 1 // b
CLOSURE L6 2 2
STORE 0 // main
LOAD 0
POP
GOTO L13
L14: NOP
LOAD 0 // h
RETURN
L13: NOP
CLOSURE L14 0 0
STORE 1 // ident
LOAD 1
POP
LOAD 0 // main
LOAD 1 // ident
GLOAD 0 // a

GLOAD 1 // b
ADD
INVOKE 2
RETURN
L3: PLOADREF 0 // a
PLOADREF 1 // b
CLOSURE L4 0 2
CONST 0
CONST 99
SUB
CONST 0
CONST 99
SUB
INVOKE 2
RETURN
L1: NOP
CLOSURE L2 2 0
CONST 9
CONST 0
INVOKE 2
HALT



CLOSURE L2 2 0

00: (CLO: L2, [], [(REF: 0), (REF: 0)]) <-TOP <-PP <-FP

CONST 9

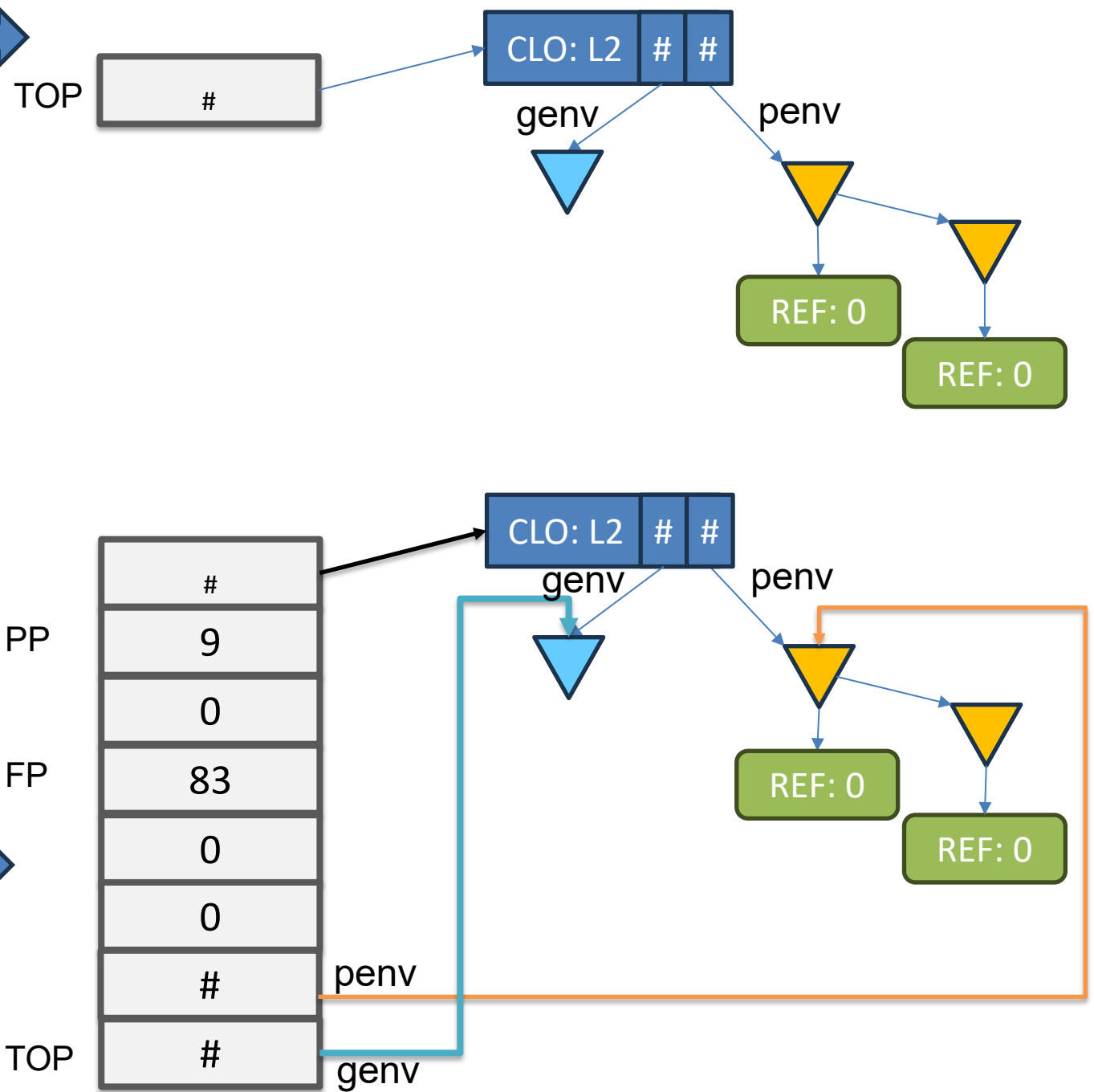
00: (CLO: L2, [], [(REF: 0), (REF: 0)]) <-PP <-FP
01: 9 <-TOP

CONST 0

00: (CLO: L2, [], [(REF: 0), (REF: 0)]) <-PP <-FP
01: 9
02: 0 <-TOP

INVOKE 2

00: (CLO: L2, [], [(REF: 0), (REF: 0)])
01: 9 <-PP
02: 0
03: 83 <-FP
04: 0
05: 0
06: [(REF: 0), (REF: 0)]
07: [] <-TOP <-end of frame

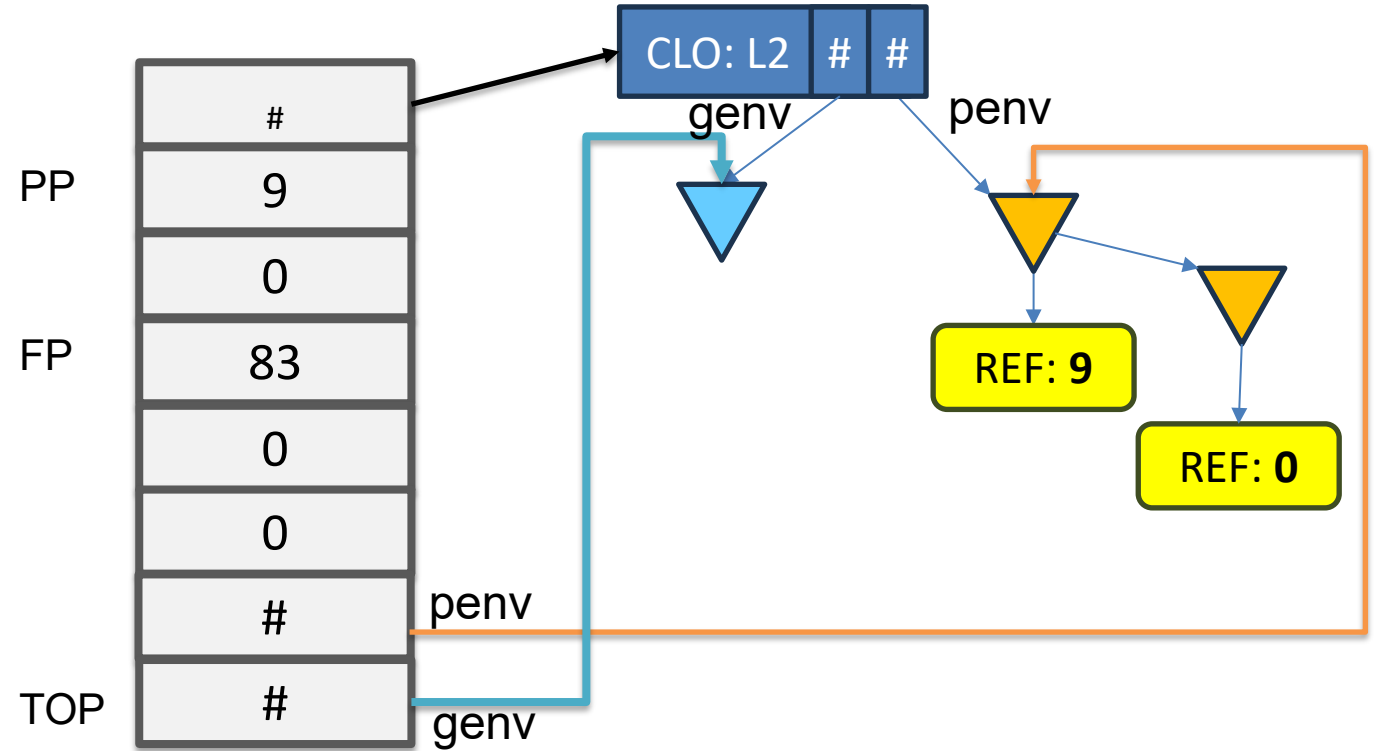


EXPORT 0 0 // a

00: (CLO: L2, [], [(REF: 9), (REF: 0)])
01: 9 <-PP
02: 0
03: 83 <-FP
04: 0
05: 0
06: [(REF: 9), (REF: 0)]
07: [] <-TOP <-end of frame

EXPORT 1 1 // b

00: (CLO: L2, [], [(REF: 9), (REF: 0)])
01: 9 <-PP
02: 0
03: 83 <-FP
04: 0
05: 0
06: [(REF: 9), (REF: 0)]
07: [] <-TOP <-end of frame

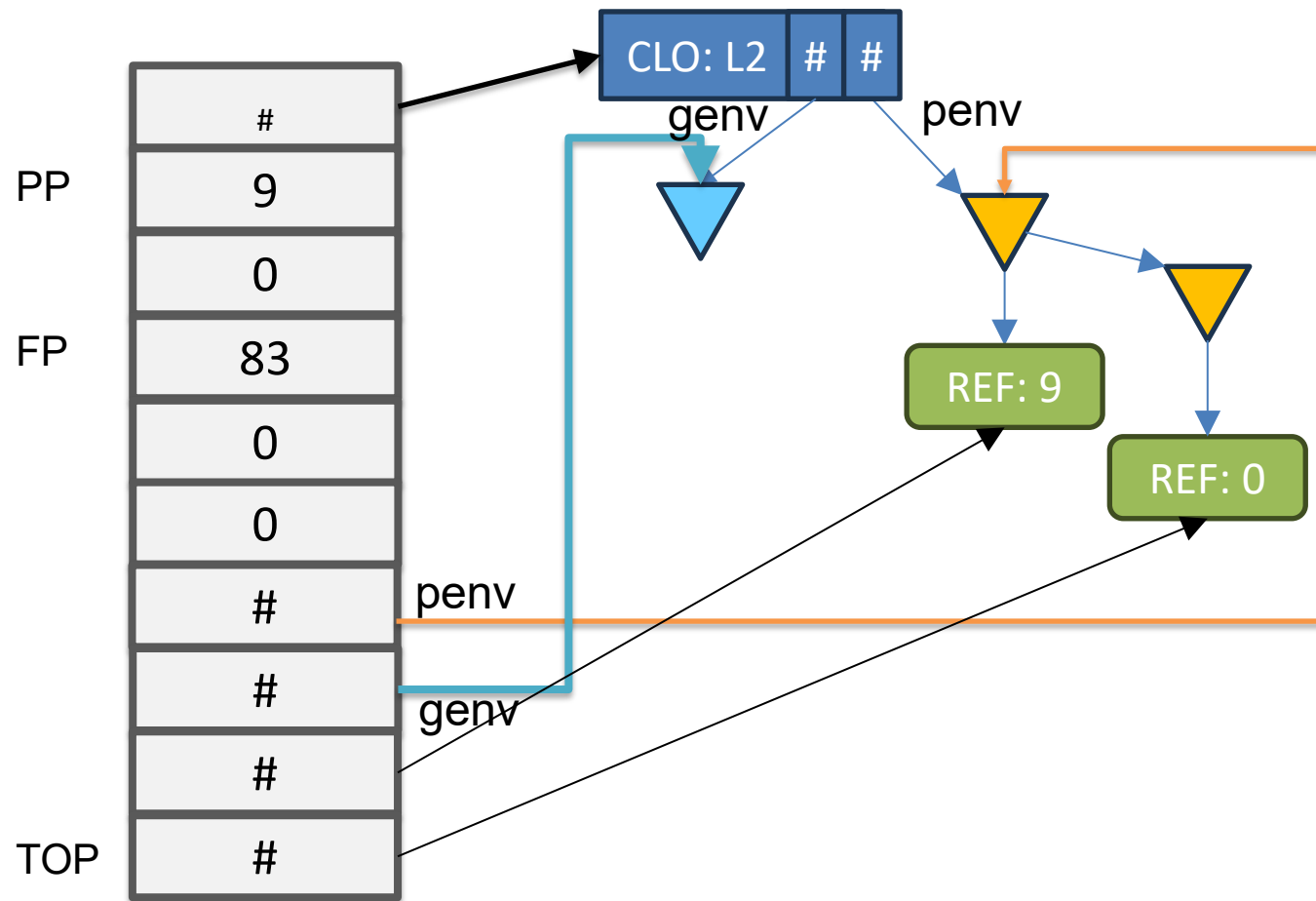


PLOADREF 0 // a

00: (CLO: L2, [], [(REF: 9), (REF: 0)])
01: 9 <-PP
02: 0
03: 83 <-FP
04: 0
05: 0
06: [(REF: 9), (REF: 0)]
07: [] <-end of frame
08: (REF: 9) <-TOP

PLOADREF 1 // b

00: (CLO: L2, [], [(REF: 9), (REF: 0)])
01: 9 <-PP
02: 0
03: 83 <-FP
04: 0
05: 0
06: [(REF: 9), (REF: 0)]
07: [] <-end of frame
08: (REF: 9)
09: (REF: 0) <-TOP



CLOSURE L4 0 2

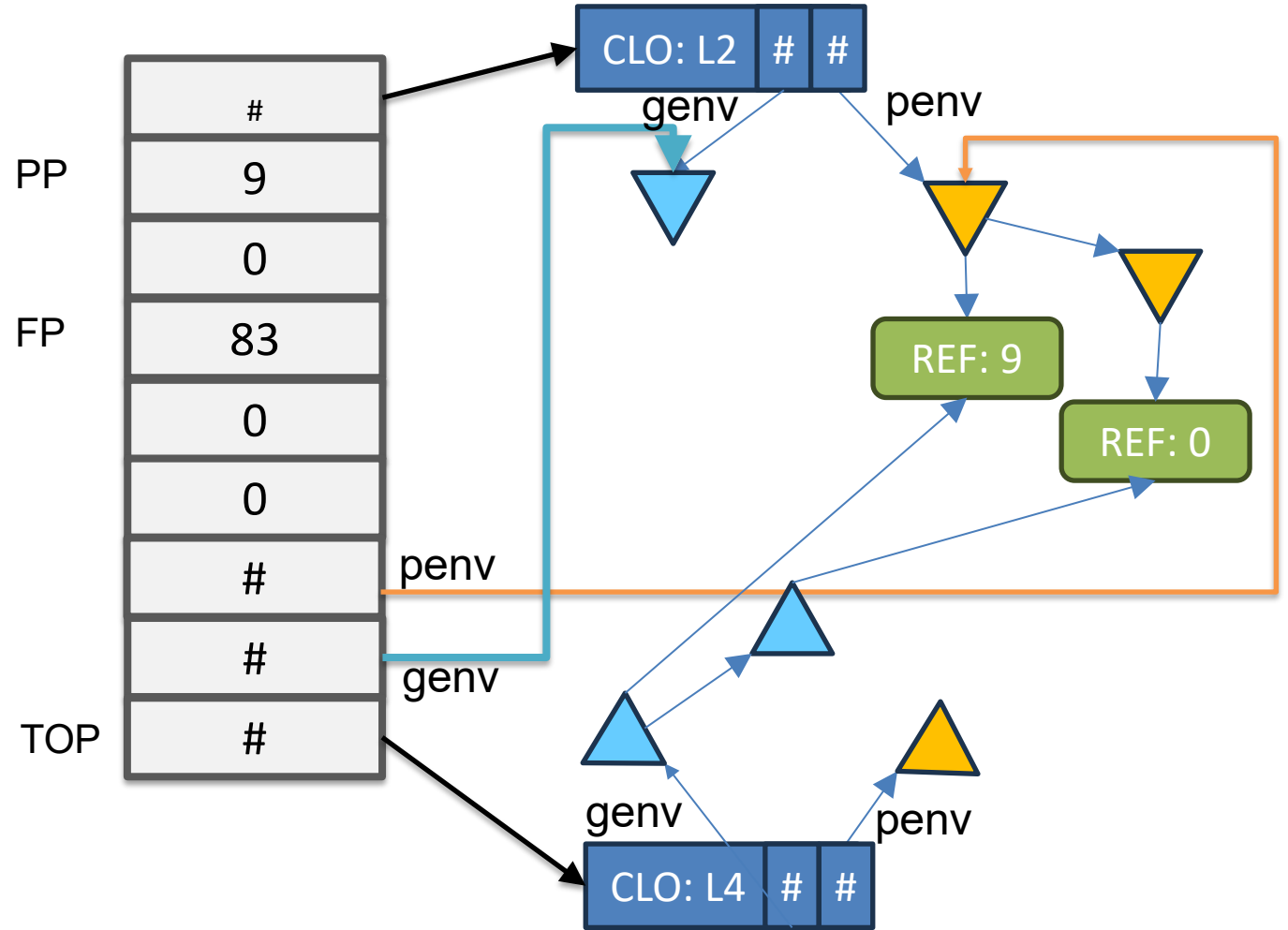
00: (CLO: L1, [], [(REF: 9), (REF: 0)])
01: 9 <-PP
02: 0
03: 83 <-FP
04: 0
05: 0
06: [(REF: 9), (REF: 0)]
07: [] <-end of frame
08: (CLO: L4, [(REF: 9), (REF: 0)], []) <-TOP

CONST 999

00: (CLO: L1, [], [(REF: 9), (REF: 0)])
01: 9 <-PP
02: 0
03: 83 <-FP
04: 0
05: 0
06: [(REF: 9), (REF: 0)]
07: [] <-end of frame
08: (CLO: L4, [(REF: 9), (REF: 0)], []) <-TOP
09: 999 <-TOP

CONST 999

00: (CLO: L1, [], [(REF: 9), (REF: 0)])
01: 9 <-PP
02: 0
03: 83 <-FP
04: 0
05: 0
06: [(REF: 9), (REF: 0)]
07: [] <-end of frame
08: (CLO: L4, [(REF: 9), (REF: 0)], []) <-TOP
09: 999
10: 999 <-TOP



INVOKE 2

00: (CLO: 1, [], [(REF: 9), (REF: 0)])

01: 9

02: 0

03: 83

04: 0

05: 0

06: [(REF: 9), (REF: 0)]

07: []

08: (CLO: 4, [(REF: 9), (REF: 0)], [])

09: 999 <-PP

10: 999

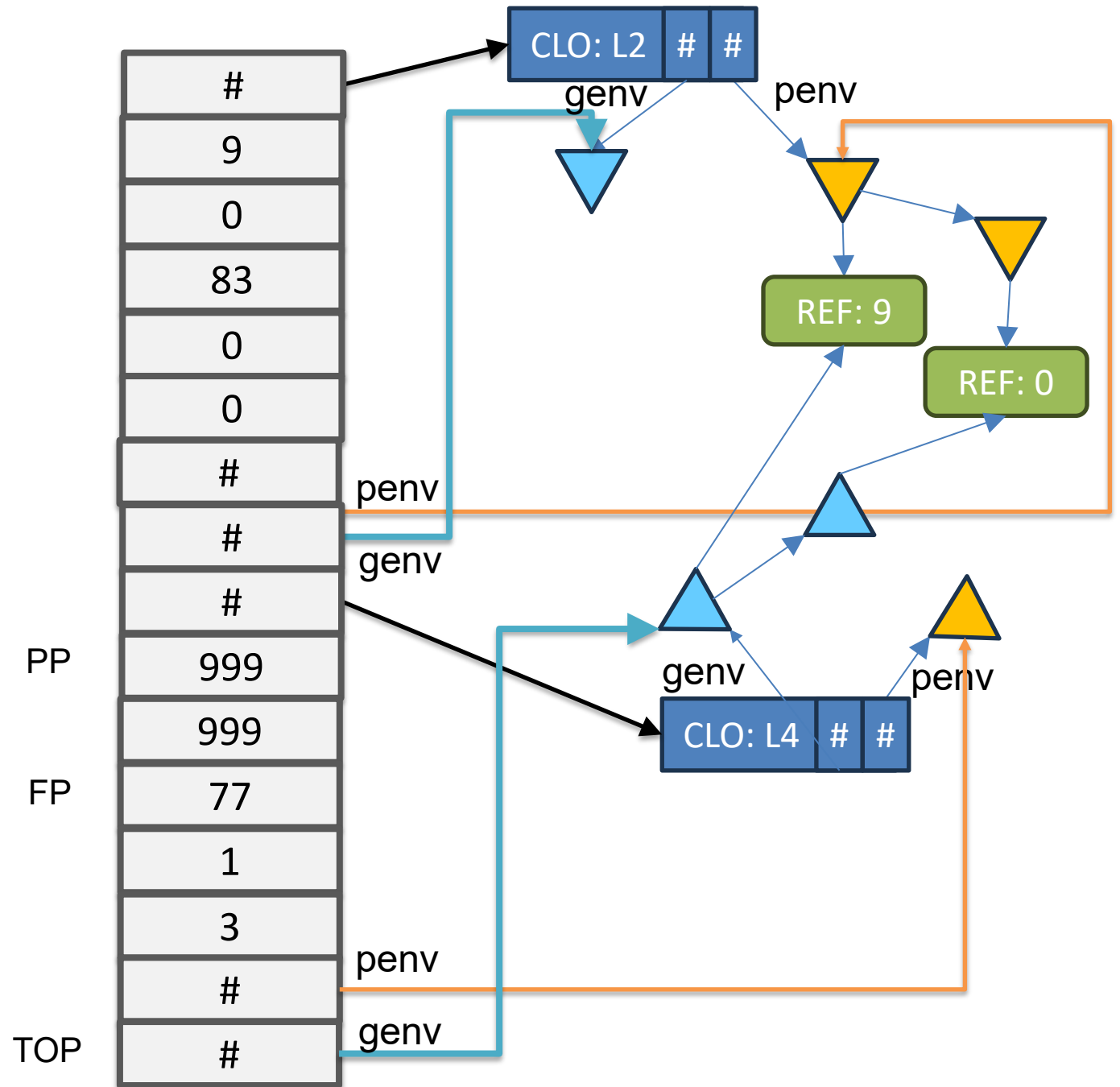
11: 77 <-FP

12: 1

13: 3

14: []

15: [(REF: 9), (REF: 0)] <-TOP <-end of frame



INVOKE 2

00: (CLO: 1, [], [(REF: 9), (REF: 0)])

01: 9

02: 0

03: 83

04: 0

05: 0

06: [(REF: 9), (REF: 0)]

07: []

08: (CLO: 4, [(REF: 9), (REF: 0)], 1)

09: 999 <-PP

10: 999

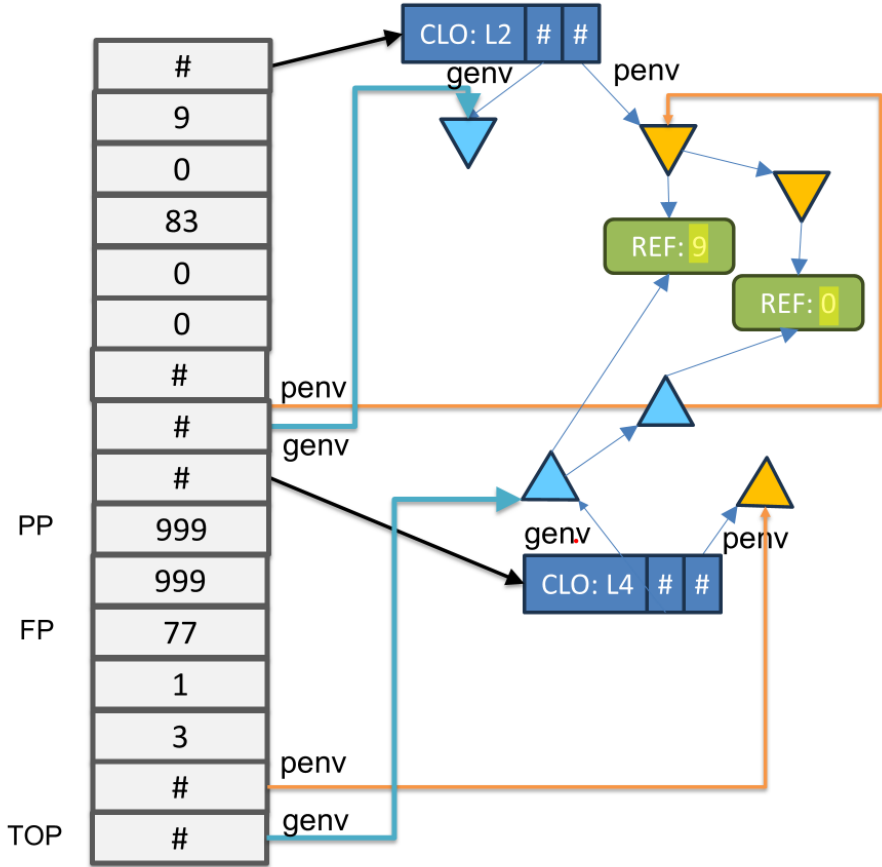
11: 77 <-FP

12: 1

13: 3

14: 1

15: [(REF: 9), (REF: 0)] <-TOP <-end of frame



GLOADREF 0 // a

00: (CLO: 1, [], [(REF: 9), (REF: 0)])
01: 9
02: 0
03: 83
04: 0
05: 0
06: [(REF: 9), (REF: 0)]
07: []
08: (CLO: 4, [(REF: 9), (REF: 0)], [])
09: 999 <-PP
10: 999
11: 77 <-FP
12: 1
13: 3
14: []
15: [(REF: 9), (REF: 0)] <-end of frame
16: (REF: 9) <-TOP

GLOADREF 1 // b

00: (CLO: 1, [], [(REF: 9), (REF: 0)])
01: 9
02: 0
03: 83
04: 0
05: 0
06: [(REF: 9), (REF: 0)]
07: []
08: (CLO: 4, [(REF: 9), (REF: 0)], [])
09: 999 <-PP
10: 999
11: 77 <-FP
12: 1
13: 3
14: []
15: [(REF: 9), (REF: 0)] <-end of frame
16: (REF: 9)
17: (REF: 0) <-TOP

CLOSURE L6 2 2

00: (CLO: 1, [], [(REF: 9), (REF: 0)])
01: 9
02: 0
03: 83
04: 0
05: 0
06: [(REF: 9), (REF: 0)]
07: []
08: (CLO: 4, [(REF: 9), (REF: 0)], []) b

STORE 0 // main

00: (CLO: 1, [], [(REF: 9), (REF: 0)])
01: 9
02: 0
03: 83
04: 0
05: 0
06: [(REF: 9), (REF: 0)]
07: []
08: (CLO: 4, [(REF: 9), (REF: 0)], [])
09: (CLO: 6, [(REF: 9), (REF: 0)], [(REF: 0), (REF: 0)]) <-PP
10: 999
11: 77 <-FP
12: 1
13: 3
14: []
15: [(REF: 9), (REF: 0)] <-TOP <-end of frame